1 Project Objectives

This project aims to develop a data-driven decision support system for independent game developers and small studios during early development planning. It uses historical game data to help users compare attributes such as genre, platform support and monetisation characteristics against platform-specific outcome groups. The intended value is a traceable reference for planning; the system does not promise future sales, revenue or commercial success.

The working scope covers Steam and Android. They are analysed separately because their source dates, available attributes and outcome definitions differ. The combined scope remains subject to supervisor confirmation. The objectives are to:

- Prepare and validate platform-specific datasets, with recorded sources, cleaning decisions and reproducible analysis inputs.
- Describe historical attribute and outcome distributions, including sample sizes, missing information and interpretation limits.
- Evaluate outcome-tier classification against simple baselines using developer-group validation and a reserved final test stage.
- Develop an application for historical condition comparison, result export and validated updates through a specified CSV format.
- Assess reliability through sensitivity and functional checks, and assess usefulness through a small user-task evaluation if feasible.

2 Proposed System Solution

The solution combines a reproducible Python analysis workflow with a browser interface. Historical Steam and Google Play snapshots are processed into separate analysis tables. Selected attributes are used for descriptive analysis and model development. The local interface currently compares two sets of planning conditions within one platform, returning matching sample counts, outcome-tier distributions and overlap information.

The product scope has been refined from analysis of a fixed dataset to an updatable application. A data maintainer can validate a CSV against a defined contract, preview exclusions, save a version and explicitly activate it. Imported application snapshots remain separate from frozen research data. Importing a file does not automatically retrain a model or alter the research split.

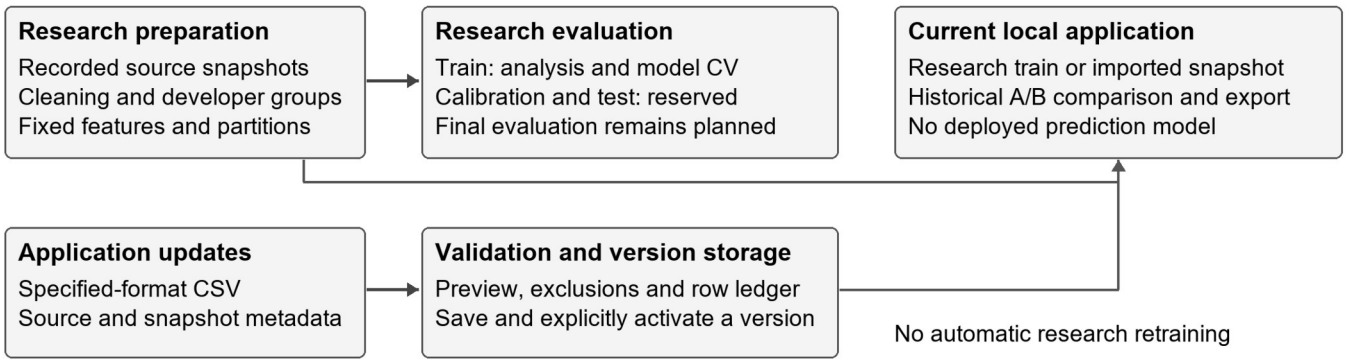


Figure 1. Research evaluation and application updates follow separate paths. The current interface provides historical comparisons; prediction deployment is not implemented.

The implementation uses Python, pandas, DuckDB and PyArrow for data handling; scikit-learn for model experiments; and HTML, CSS and JavaScript for the local interface. Power BI provides supporting descriptive views. The planned website will bring the overview, data exploration, comparison and research results into a coherent interface.

3 Work Completed

3.1 Data preparation and analysis design

The two principal inputs are the Steam Games Dataset 2025 March snapshot (Ermilov, 2025) and the Google Play Store Apps June 2021 collection (Prakash, 2021). Other downloaded sources were audited for comparison and feasibility; they were not all merged into the primary analysis. Original files are retained, while intermediate and processed tables are stored separately.

Completed preparation includes source inventories, structural and data-quality checks, conservative parsing, row-level inclusion records and a fixed feature policy. Missing values are distinguished from zero or false. Literal game names such as None and NULL are retained for review rather than automatically removed. Inconsistent Android free/price records have an unknown free status in the analysis version; non-USD or unknown currencies are not treated as USD.

The Steam primary sample excludes nonpositive observed prices, unavailable targets or developer groups, and software-only genre records under an explicit heuristic. Android uses 16 clearly identified game categories and excludes ambiguous Sports records. These choices define the scope of the findings and do not establish representativeness of the entire market.

Table 1. Fixed primary analysis cohorts and developer-group partitions.

Platform	Total	Train	Calibration	Test
Steam	73,527	51,152	11,435	10,940
Android	318,300	221,806	48,297	48,197

Steam labels are three groups mapped from estimated-owner intervals; an owners interval of 0-0 is treated as unknown. Android labels use installation lower bounds: below 1,000, 1,000 to below 100,000, and at least 100,000. Neither target is a measure of revenue or exact sales.

Developers are grouped before partitioning, so the same developer group does not cross train, calibration and test sets or the three training validation folds. Steam co-development links are included when forming groups. Learned encoding, imputation and scaling are fitted only on the fitting portion of each training fold. Outcome fields and developer identifiers are excluded from predictors. These controls address leakage risks rather than guaranteeing an unbiased model (Kapoor and Narayanan, 2023).

3.2 Descriptive analysis and verification

Initial training-set analysis covers outcome balance, genres or categories, monetisation attributes, observation age and missing information. Verified exports support two Power BI historical-overview pages. The original application version uses only training rows: 51,152 Steam and 221,806 Android games. Source files, outputs and experiment settings have recorded hashes for version checks; independent verification scripts check row membership, counts and model outputs. The latest completed research stage recorded 48 passing unit and integration tests. These checks are engineering evidence, not user evaluation.

3.3 Development model results

The completed experiments compare a class-prior baseline, age-only models, Logistic Regression and Random Forest. P denotes planning-describable snapshot attributes; P+A adds the elapsed time between release and observation. P is not a set of verified pre-release measurements. A Steam variant without price was also evaluated because the original price currency remains unresolved.

Table 2. Mean macro-F1 over three training developer-group folds. These are development results, not final test scores.

Platform	Features	Prior baseline	Logistic regression	Random forest
Steam	P	0.2860	0.4401	0.4458
Steam	P+A	0.2860	0.4845	0.5062
Android	P	0.2127	0.4758	0.4785
Android	P+A	0.2127	0.5322	0.5376

Macro-F1 averages the F1 score across outcome classes and is useful when class sizes differ. The prior baseline is the same within each platform because it does not use P or P+A attributes. All four displayed learned-model comparisons exceed that baseline in development CV. Random Forest has a larger improvement for Steam P+A than for Android P, where the improvement is small. These comparisons reuse the folds used to select settings and therefore are not independent estimates of final performance.

Additional fixed-setting experiments restrict observation age, vary Steam sample eligibility, and test four Android outcome tiers. Some model-family rankings change with sample scope, and higher-install Android classes remain difficult. The original three-tier product behaviour is retained. Final model choice, calibration and reserved-test evaluation have not been completed, and no predictive probabilities have been deployed.

3.4 Application prototype

The local prototype supports Steam/Android selection, two-condition comparison, small or empty sample feedback, and JSON export containing conditions and data provenance. CSV validation, preview, version saving, activation and return to the original research version are implemented. The saved active version persists between runs. The application accepts a specified schema rather than arbitrary CSV files.

The comparison prototype and CSV workflow have recorded automated and browser checks. A Power BI report and two overview screenshots have also been prepared. Figure 2 illustrates the Android training-sample overview; Power BI is supporting material rather than a required dependency of the application. The saved PBIX interactions and DAX have not been independently verified.

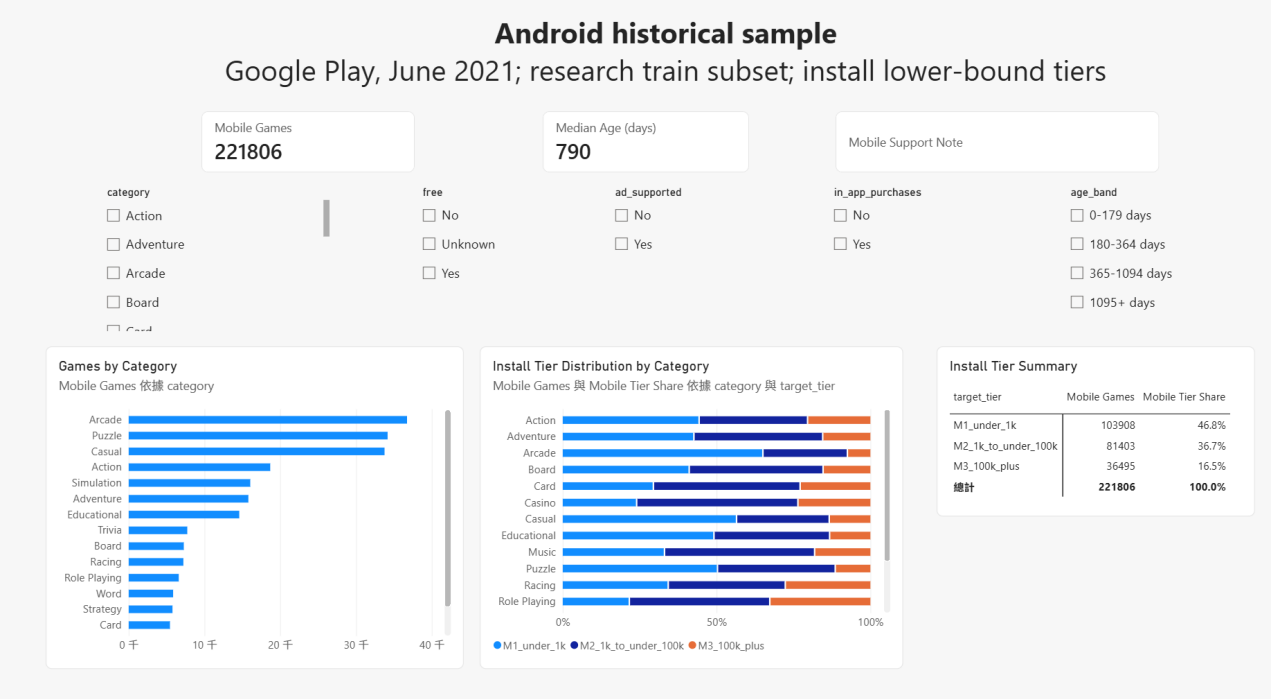


Figure 2. Existing Android Power BI overview screenshot, captured on 16 October 2026. Its 221,806-game denominator is the research training subset, not the full Android market.

4 Problems and Mitigation

Issue	Response and remaining work
Different snapshots and outcome meanings	Analyse Steam and Android separately; display source dates and scope. Do not interpret owner intervals or install lower bounds as revenue or combine them into a cross-platform success ranking.
Selection, missingness and price uncertainty	Retain row-level reasons and explicit unknown values. Report primary-sample restrictions; use the completed scope and without-price checks when interpreting results.
Limited predictive reliability	Retain simple baselines and inspect class/subgroup results. Complete the reserved evaluation before any prediction deployment; historical comparison remains useful without a deployed classifier.
Scope and delivery risk	Seek confirmation of the dual-platform direction. Prioritise the existing comparison/import workflow and a clear analysis interface. Limit optional features until the core workflow and its usefulness have been evaluated.

5 Schedule and Current Activities

The recorded work has progressed through data audit and preparation on 5-6 October, initial analysis and linear-model development on 7-8 October, CSV/version support and tree experiments on 10 October, sensitivity checks completed by 13 October, and Power BI material and program guidance by 17 October. The current priority is to integrate these components into a clearer demonstration and communicate their limitations. The updated plan below prioritises a working interface, completion of the research evaluation and preparation for assessment.

Table 3. Proposed work packages after this report. Timing is a working estimate and will be aligned with the supervisor-confirmed assessment dates.

Window	Planned work	Completion evidence
Next 1-2 weeks	Confirm scope; implement analysis overview and searchable, sortable, paginated data exploration; connect existing comparison and import workflows.	Coherent local demonstration with traceable counts and a documented user task.
Weeks 3-4	Complete group-versus-random diagnostic; extend relevant literature review; define and implement feasible similar-game references.	Recorded comparison protocol/results and clearly explained matching behaviour.
Weeks 5-6	Fix model choices; refit on train, calibrate and evaluate the reserved test set in sequence. Decide whether prediction is suitable for the interface.	Final evaluation with per-class and subgroup results; explicit deployment decision.
Weeks 7-8	Complete integration and functional checks; conduct a small user-task study if feasible; prepare presentation, setup guide and report updates.	Test evidence, documented user feedback where obtained, and a rehearsed demonstration.

Current activities centre on reporting verified progress, reviewing the system structure and preparing the next interface work. The overview/explorer sections, similarity retrieval and final model evaluation are planned work, not completed functions. Public deployment will be considered after local integration; the present Python service is a local prototype. Documentation and supervisor feedback will be maintained throughout the remaining work.

References

Ermilov, Artemiy. 2025. Steam Games Dataset 2025. Kaggle dataset, March 2025 snapshot. https://www.kaggle.com/datasets/artermiloff/steam-games-dataset

Kapoor, Sayash, and Arvind Narayanan. 2023. Leakage and the reproducibility crisis in machine-learning-based science. Patterns 4, 9, Article 100804. https://doi.org/10.1016/j.patter.2023.100804

Prakash, Gautham. 2021. Google Play Store Apps. Kaggle dataset, June 2021 collection. https://www.kaggle.com/datasets/gauthamp10/google-playstore-apps

Supporting Project Records

The reported measurements and implementation checks are documented in the following records retained with the project:

- Data scope and partitions: Program/reports/2026-10-06/analysis_v1/manifest.json, analysis_protocol.md and verification.json.
- Initial data analysis and linear models: Program/reports/2026-10-07/train_eda_v1/ and model_dev_v1/.
- Tree comparison: Program/reports/2026-10-10/model_tree_v1/linear_tree_comparison.csv and verification.json.
- Age and sample/target sensitivity: Program/reports/2026-10-11/model_age_v1/ and reports/2026-10-13/model_scope_v1/.
- Application and import checks: Program/DEMO_GUIDE.md, CSV_IMPORT_GUIDE.md and reports/2026-10-10/csv_import_v1/verification_notes.md.
- Power BI material: Program/reports/powerbi/. AI assistance record: Program/reports/planning/ai_assistance_log.md.

Large source files, processed data and saved model artifacts are retained locally and are not all included in Git. The repository documentation explains the required data and environment; a code-only clone is not a complete reproduction package.